Worksheet 13 — Cloud & Container Security (4 hrs)

> **Course:** Software Security (KOSEN69) · Week 13 **Aligned to:** OWASP 2025 **A02 Security Misconfiguration** · **CWE-732** (incorrect permission assignment), **CWE-16** (configuration), plus CWE-798, CWE-250, CWE-538, CWE-269, CWE-1104 **Signature game:** 🔍 *Misconfig Hunt* (CloudGoat-style) — each misconfig you find **and** fix = a flag.

> ⚠️ **Ethics note:** All artifacts here are deliberately broken for teaching and are marked "*Sandbox/teaching only; for authorized lab use.*" Scan, exploit, and harden **only** these lab files (or systems you own / are authorized to test). Never point Trivy or these techniques at third-party cloud accounts or images without written permission.

Part 1 — Student Information

Name	Student ID	Date	Group	AI use
Andrew Dicesare	6631503050	10/26/2026	Let me eat	AI assistant used as a working aid only — full disclosure in the note below this table

AI use, in full: I used an AI assistant to help draft and refine the written answers — the Part 2 lecture questions, the Part 4 reflection, the mitigation notes, and the **Audit the AI** / **Prompt Problem** sections (which quote the AI by design). I ran the lab myself (the Trivy `config` scan) and captured every screenshot; the findings I mapped, the CWE reasoning, and the conclusions are mine. I also had the assistant run `scan.sh` independently to cross-check the baseline finding count before finalising.

Part 2 — Lecture Questions

Answer in your own words (2–4 sentences each).

1. Explain the **shared-responsibility model**. In `harden.md` it says "the cloud provider secures *of* the cloud; you secure what you put *in* it." Give one concrete example of each side for an S3-backed app.

   A: The model splits security between provider and customer: the cloud provider is responsible for security **of** the cloud — the physical data centres, the storage hardware and hypervisor, the network backbone, and the durability/availability of the S3 service itself — while the customer is responsible for security **in** the cloud, i.e. how they configure and use it. For an S3-backed app: the **provider side** is AWS patching the storage infrastructure and guaranteeing the S3 service runs and doesn't lose your objects; the **customer side** is you turning on Block Public Access, writing the bucket policy and IAM roles that decide who can read/write, enabling encryption, and not uploading secrets. A publicly-exposed bucket is a customer misconfiguration (A02), not an AWS failure.

2. `iam-policy-insecure.json` uses `"Action": "*"` and `"Resource": "*"`. Describe **why this violates least privilege** and map it to CWE-732 vs CWE-269 — what is the difference between the two CWEs here?

A: The app only needs to *read objects from one bucket prefix*, but `Action:"*"` + `Resource:"*"` grants **every** action on **every** resource — full admin — so a stolen credential or a compromised app can delete buckets, create IAM users, launch EC2, or read every secret in the account. That is the opposite of least privilege (grant the minimum action, on the minimum resource, under the right condition). The two CWEs sit on different axes: `Action:"*"` → **CWE-269 (improper privilege management)** — the *identity* holds far more privileges (capabilities) than it needs; `Resource:"*"` → **CWE-732 (incorrect permission assignment for a critical resource)** — the permission is *assigned over far more resources* than it should reach. In short, CWE-269 is "can do too many things," CWE-732 is "can touch too many things."

3. The insecure Dockerfile bakes `ENV API_TOKEN=sk_live_...`. Explain how an attacker recovers that secret from a shipped image (name the `docker` commands) and why this is **CWE-798 / CWE-200**.

   A: A Docker image is just stacked, readable layers plus a config blob, and `ENV` values are stored in that metadata — so the attacker never even has to run the container. With the image pulled, `docker history --no-trunc <image>` prints every build instruction including the `ENV API_TOKEN=sk_live_...` line, and `docker inspect <image>` shows the full `Config.Env` array with the token value in clear text. This is **CWE-798 (use of hard-coded credentials)** because a real credential is embedded in the artifact, and **CWE-200 (exposure of sensitive information)** because the image is distributed, so anyone who can pull it reads the secret. The same applies to the baked `AWS_SECRET_ACCESS_KEY`.

4. Why is `FROM python:latest` (CWE-1104/CWE-16) a security and reproducibility problem, and how does pinning to an `@sha256:` digest fix it?

   A: `:latest` is a **mutable** tag — it points to whatever the maintainer most recently pushed, so the same `docker build` run today and next month can pull different underlying images with different Python patches, OS packages, and CVEs. That is a **security** problem (you can't know or audit what's in your base, and a changed/compromised `latest` silently alters your supply chain — CWE-1104/CWE-16) and a **reproducibility** problem (builds aren't deterministic, so you can't rebuild or audit a past release). Pinning to `FROM python:3.11-slim@sha256:<digest>` fixes it because a digest is an **immutable content hash**: it always resolves to the exact same bytes, so the base is reproducible and you control precisely which version ships — upgrading becomes a deliberate digest bump you can review and scan, not silent drift.

5. What does a **distroless** base image remove (relative to `python:3.11-slim`), and how does that shrink the attack surface for an RCE or container-escape attacker?

   A: A distroless image (e.g. `gcr.io/distroless/python3-debian12`) ships only the app and its runtime — the Python interpreter and libraries — and **removes the shell** (`/bin/sh`, `bash`), the **package manager** (apt/apk), coreutils and network tools (`ls`, `cat`, `curl`, `wget`), and most of the OS userland. That shrinks the attack surface for an RCE/container-escape attacker because once they have code execution they have **no shell to spawn, no package manager to install tools, and no curl/wget to pull a second-stage payload**, and there are far fewer binaries to exploit or "live off the land" with — many standard post-exploitation steps simply aren't available. Fewer packages also means dramatically fewer CVEs in the image to begin with.

Part 3 — Hands-on Lab (180 min)

Learning goals: find cloud/container misconfigurations with Trivy, map each to a CWE, and rebuild the IAM policy + Dockerfile to least privilege / hardened so the findings drop.

Prerequisites: Docker installed and running; the `week13-cloud-container/` lab folder; internet access to pull `aquasec/trivy:latest`.

> **How this section is filled in:** each task gives the command(s), the result, the CWE mapping, and a mitigation note. `‹…›` marks a real count/line to capture, and each 📸 marker is a stamped screenshot to take.

Environment setup (real commands)

```
cd labs/week13-cloud-container
bash scan.sh                                  # trivy config over both Dockerfiles
(IAM reviewed manually)
# or directly (MEDIUM kept in on purpose — DS-0001 :latest is MEDIUM):
docker run --rm -v "$PWD:/src" aquasec/trivy:latest config --severity
MEDIUM,HIGH,CRITICAL /src
# optional: build + CVE-scan the hardened image
docker build -f Dockerfile.hardened -t week13-hardened:lab .
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest
image --severity HIGH,CRITICAL week13-hardened:lab
```

Same app, shipped twice — the scanner sees only three rows

OWASP 2025 A02 Security Misconfiguration · trivy config --severity MEDIUM,HIGH,CRITICAL

flagged by trivy config · found only by reading the file

INSECURE IMAGE (Dockerfile.insecure)		HARDENED IMAGE (Dockerfile.hardened)
FROM python:latest mutable tag — rebuild ≠ same image **SCANNER · Trivy DS-0001**	**Base image tag** CWE-1104 · CWE-16	FROM python:3.11-slim@sha256:cdbd05... multi-stage: build stage is discarded digest pin — same bytes every rebuild
ENV API_TOKEN=sk_live_supersecret_... docker history prints it back **SCANNER · Trivy DS-0031**	**Baked-in secret** CWE-798 · CWE-200	no ENV secret in any layer AWS_SECRET_ACCESS_KEY gone too injected at runtime, never baked in
no USER — runs as root uid 0: RCE or escape starts as root **SCANNER · Trivy DS-0002**	**Runtime user** CWE-250	USER 65532:65532 the distroless "nonroot" uid an escape lands unprivileged
COPY . . .git, .env, keys, CI files ride along **READ THE FILE · no Trivy rule**	**Build context** CWE-538	COPY app.py . plus a .dockerignore only the file the app actually needs
RUN chmod -R 777 /app any process can rewrite the code **READ THE FILE · no Trivy rule**	**Directory perms** CWE-732	default perms, not world-writable run: --read-only --cap-drop ALL nothing inside can rewrite app.py
RUN pip install -r requirements.txt unpinned, as root, no --no-cache-dir **READ THE FILE · no Trivy rule**	**Dependency install** CWE-1104	venv built in a stage, then copied distroless runtime: no pip, no shell nothing left to call after an RCE

Same argument one layer up — the IAM policy the container assumes

iam-policy-insecure.json		iam-policy-leastpriv.json
"Action": "*" "Resource": "*" no Condition — nothing narrows it **READ THE FILE · no Trivy rule**	**IAM policy** CWE-269 · CWE-732 = full AWS admin	"Action": ["s3:GetObject"] on arn:aws:s3:::lab-app-bucket/app/* + Condition s3:prefix = app/* one action, one prefix, one condition

trivy config flags 3 rows. The other 4 you find by reading.

DS-0001 the tag · DS-0002 the root user · DS-0031 the ENV secret.
COPY . . , chmod 777, pip and the IAM JSON have no rule at all.
Task 1 grades that split: the rule ID, or "manual review".

Task 0 — Onboarding (15 min)

Confirm Docker works, run `bash scan.sh`, record the **baseline finding count** against `Dockerfile.insecure`.

Commands:

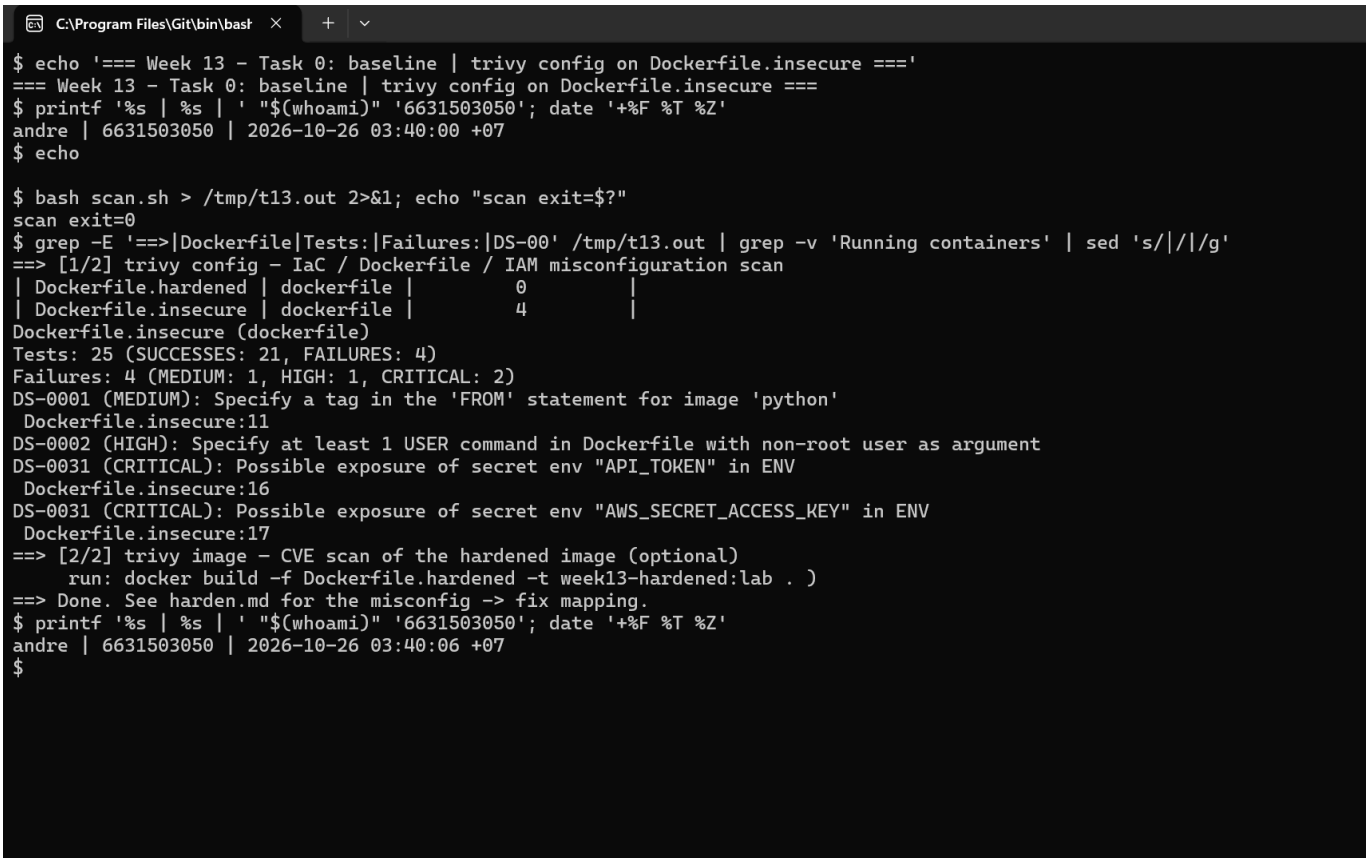
```
docker version
bash scan.sh
```

Baseline: `trivy config` reports **4** misconfigurations against `Dockerfile.insecure` (MEDIUM: 1, HIGH: 1, CRITICAL: 2) — DS-0001 `:latest`, DS-0002 root user, and **DS-0031 secret-in-`ENV` firing twice** (once each for `API_TOKEN` and `AWS_SECRET_ACCESS_KEY`). `Dockerfile.hardened` in the same scan reports **0**. (Trivy's `config` scanner does **not** parse standalone IAM JSON, so `iam-policy-insecure.json` is reviewed manually in Task 2.)

```
Report Summary
┌──────────────────────┬────────────┬───────────────────┐
│        Target        │    Type    │ Misconfigurations │
├──────────────────────┼────────────┼───────────────────┤
│ Dockerfile.hardened  │ dockerfile │         0         │
│ Dockerfile.insecure  │ dockerfile │         4         │
└──────────────────────┴────────────┴───────────────────┘

Dockerfile.insecure (dockerfile)
Tests: 25 (SUCCESSES: 21, FAILURES: 4)
Failures: 4 (MEDIUM: 1, HIGH: 1, CRITICAL: 2)

DS-0001 (MEDIUM):   Specify a tag in the 'FROM' statement for image 'python'   →
Dockerfile.insecure:11
DS-0002 (HIGH):     Specify at least 1 USER command with a non-root user
DS-0031 (CRITICAL): Possible exposure of secret env "API_TOKEN" in ENV          →
Dockerfile.insecure:16
DS-0031 (CRITICAL): Possible exposure of secret env "AWS_SECRET_ACCESS_KEY"     →
Dockerfile.insecure:17
```



Task 1 — Misconfig Hunt: container image (40 min)

Find every defect in `Dockerfile.insecure`; only 3 of 6 map to a Trivy rule — identify the rest by code review.

Defect table (6 rows):

#	Defect (line in `Dockerfile.insecure`)	CWE	Trivy rule / severity
1	`FROM python:latest` — unpinned, mutable base	CWE-1104 / CWE-16	**DS-0001** (":latest" tag) · MEDIUM
2	Runs as root — no `USER` directive	CWE-250	**DS-0002** (root user) · HIGH
3	`ENV API_TOKEN=...` / `AWS_SECRET_ACCESS_KEY=...` — secret in image	CWE-798 / CWE-200	**DS-0031** (secret in ENV) · CRITICAL · **fires twice** (one per secret ENV)
4	`COPY . .` — copies whole build context (.git/.env/keys)	CWE-538	manual review — no Trivy rule
5	`chmod -R 777 /app` — world-writable app dir	CWE-732	manual review — no Trivy rule
6	`RUN pip install -r requirements.txt` — unpinned install	CWE-1104	manual review — no Trivy rule

Why only 3 map: Trivy's Dockerfile `config` scanner checks declarative instruction *configuration* (base-image tag, final `USER`, secret-looking `ENV`). `COPY . .`, `chmod -R 777`, and an unpinned `pip install` are RUN/COPY *semantics* it doesn't evaluate, so they're caught by code review against the `# DEFECT:` comments and `harden.md`.

Task 2 — Misconfig Hunt: IAM (30 min)

Document why `iam-policy-insecure.json` is full-admin.

#	**Misconfiguration (`iam-policy-insecure.json`)**	**CWE**	**Fix**
7	`"Action": "*"` — every action	CWE-269	Enumerate only the actions used (e.g. `s3:GetObject`, `s3:ListBucket`)
8	`"Resource": "*"` — every resource	CWE-732	Scope to specific ARNs (one bucket + prefix)
9	No `Condition` block	CWE-732	Add a `Condition` (e.g. `s3:prefix`, source IP, MFA)

Blast radius if these creds leak: the policy is `Allow Action:"*" Resource:"*"` = **full administrator over the entire AWS account**. A leaked credential could read and delete every S3 bucket, create or escalate IAM identities (persistence), spin up EC2 for crypto-mining, exfiltrate all data, and delete backups/CloudTrail to cover tracks — total account takeover, not a single-app incident. Least privilege would cap the damage to read-only access to one bucket prefix.

Task 3 — Secrets & storage (25 min)

Show secrets must not live in the image.

Proof — the baked secret leaks straight from the built image (no container run needed):

```
docker build -q -f Dockerfile.insecure -t week13-insecure:lab .
docker history --no-trunc --format '{{.CreatedBy}}' week13-insecure:lab | grep -E
'API_TOKEN|AWS_SECRET_ACCESS_KEY'
```

Captured output — both secrets sit in clear text in the image layers:

```
ENV AWS_SECRET_ACCESS_KEY=AKIAEXAMPLEdeadbeefdeadbeef
ENV API_TOKEN=sk_live_supersecret_DO_NOT_DO_THIS
```

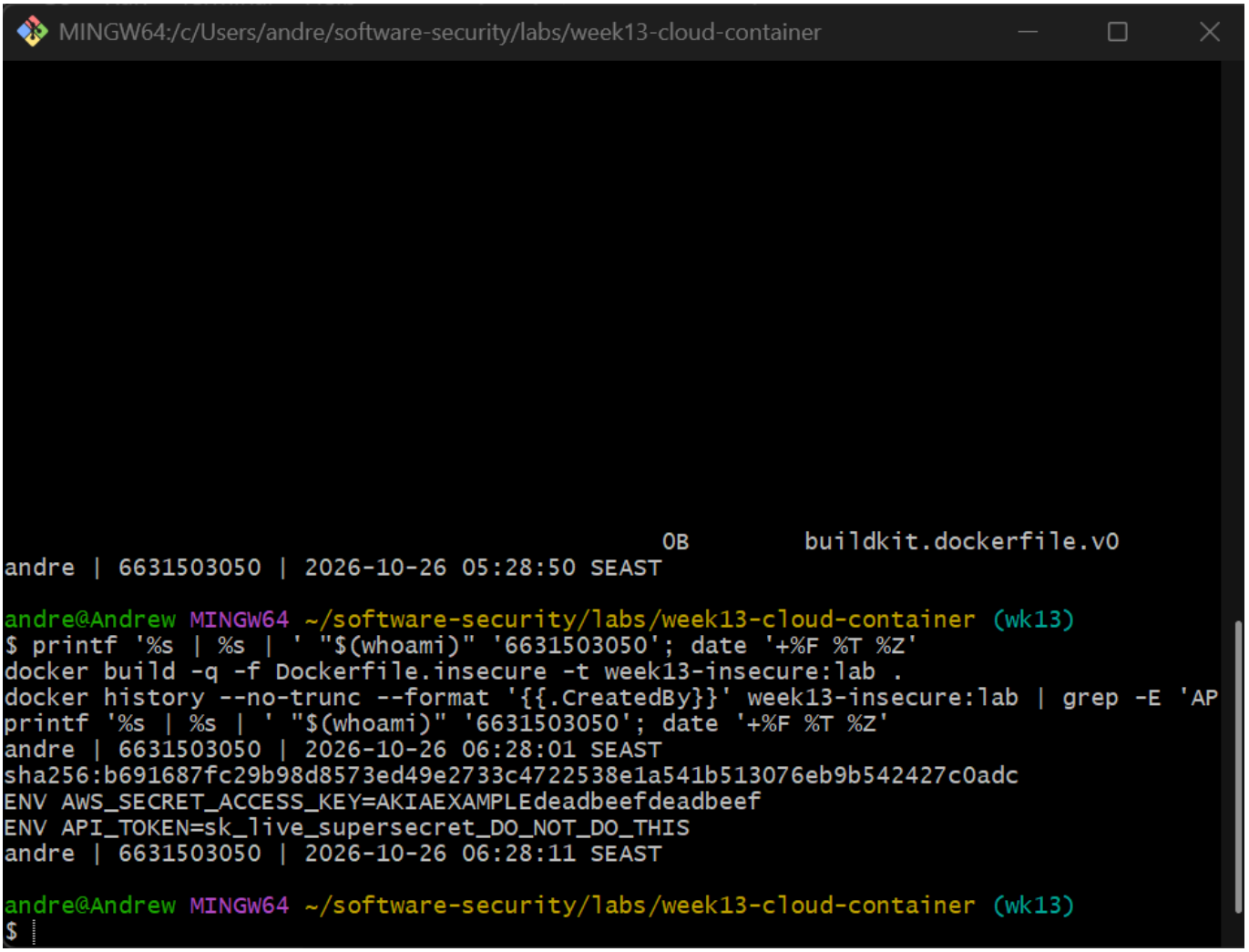
(`--format '{{.CreatedBy}}'` prints just the layer instruction, so the two `ENV` secret lines are clean; plain `docker history --no-trunc` shows the same thing padded across wide columns.) `docker inspect week13-insecure:lab` likewise lists both under `Config.Env`. Anyone who can pull the image reads the live keys **without ever starting it** — CWE-798 (hard-coded credentials in the artifact) / CWE-200 (exposure of sensitive information).

Remediation (2–3 sentences): `ENV API_TOKEN` and `ENV AWS_SECRET_ACCESS_KEY` are baked into the image config and are readable by anyone who can pull it via `docker history --no-trunc` / `docker inspect` (`Config.Env`) — no running container needed (CWE-798/200). They must be **injected at runtime instead of built in**: from a secrets manager (AWS Secrets Manager / SSM Parameter Store, fetched at startup via the

app's IAM role), a mounted secret file (Kubernetes/Docker secret), or an orchestrator secret — so the value never becomes an image layer. For local runtime injection: `docker run -e API_TOKEN=...` or `--env-file`, never `ENV`/`--build-arg`.

`.dockerignore` entries to stop `COPY . .` leaking (CWE-538):

```
.git
.gitignore
.env
*.pem
*.key
secrets/
.aws/
__pycache__/
*.pyc
node_modules/
*.log
Dockerfile*
.dockerignore
```



Task 4 — Hardened-Dockerfile + least-priv-IAM defense (50 min)

Prove the "after" is clean.

(a) Before / after finding counts (`trivy config`):

Artifact	Findings (MEDIUM+)
`Dockerfile.insecure`	**4** (DS-0001 MEDIUM, DS-0002 HIGH, DS-0031 CRITICAL ×2) + 3 manual-review defects
`Dockerfile.hardened`	**0** (clean — the digest pin also clears DS-0001 on the distroless `FROM`)

(b) Defect → fix mapping (`Dockerfile.hardened`):

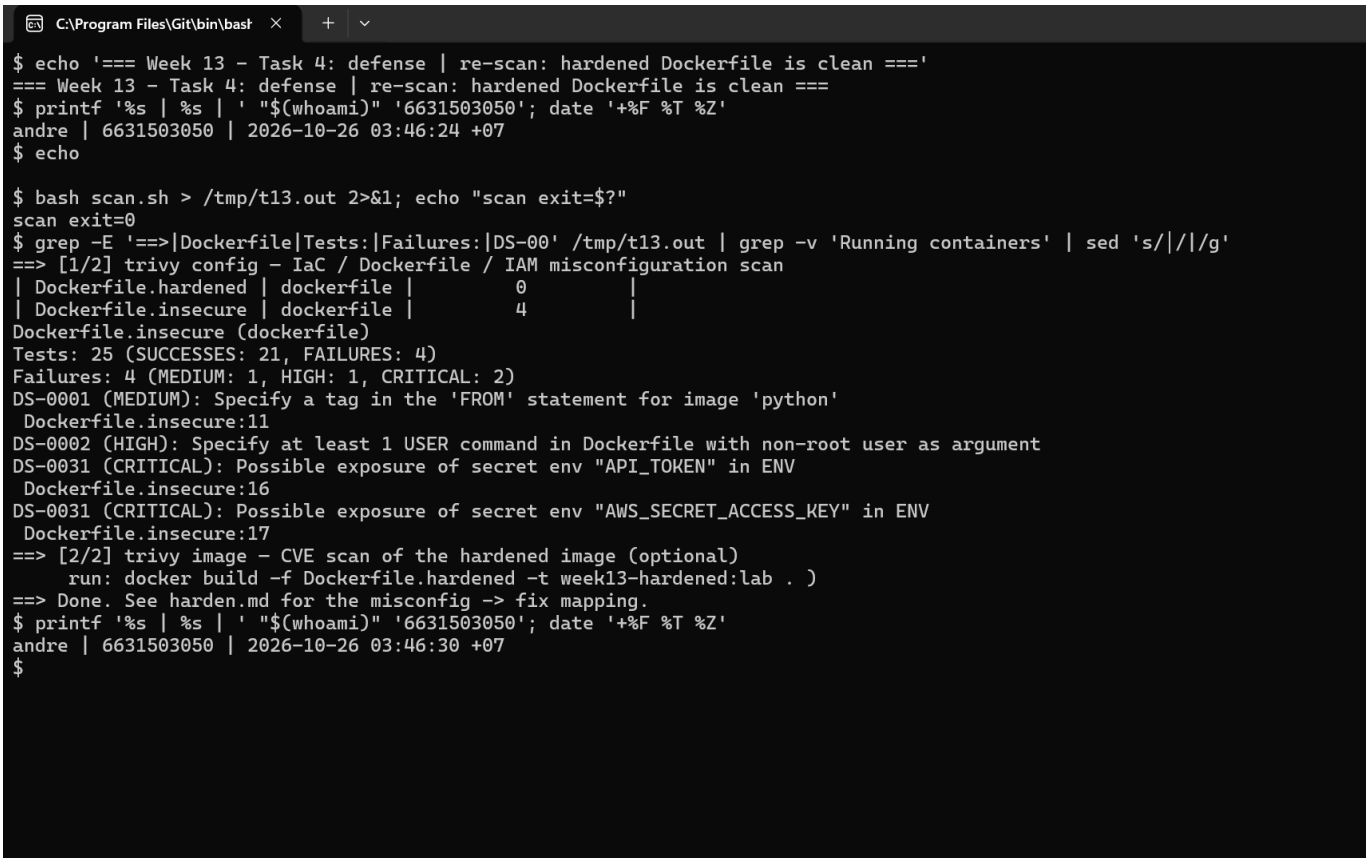
Task-1 defect	Fix in `Dockerfile.hardened`	Why it reduces risk
`FROM python:latest`	`FROM python:3.11-slim@sha256:…` (build) + distroless `@sha256:…` (runtime)	Immutable, reproducible, auditable base
Secret in `ENV`	No secret in the image — injected at runtime	Nothing to recover via `docker history`/`inspect`
`COPY . .`	`COPY requirements.txt .` then `COPY app.py .` (+ `.dockerignore`)	Only app code ships; no `.git`/`.env` leak
`chmod -R 777`	Default perms; app not world-writable	No in-container tampering with code
Runs as root	`USER 65532:65532` (distroless `nonroot`)	RCE/escape no longer lands as root
Unpinned `pip install`	Pinned `requirements.txt` installed into a venv in a throwaway build stage	Reproducible deps; build tools not in runtime

(c) IAM rewrite (`iam-policy-leastpriv.json`): scopes `s3:GetObject` to `arn:aws:s3:::lab-app-bucket/app/*`, limits `s3:ListBucket` to the bucket ARN with a `Condition StringLike s3:prefix: ["app/*"]`, and drops `Action:"*"`/`Resource:"*"`. The app keeps exactly what it needs (read its objects under one prefix); everything else is denied by default.

(d) Extra runtime hardening (from `harden.md`, enforced at deploy):

```
docker run --read-only --cap-drop ALL --security-opt no-new-privileges \
           --memory 256m --pids-limit 100 week13-hardened:lab
```

`--read-only` (immutable FS), `--cap-drop ALL` (no Linux capabilities), `--security-opt no-new-privileges` (no setuid escalation); never `--privileged` or Docker-socket mounts.



Task 5 — Flag tally (20 min)

Flags: 9 / 9 — 6 container (`:latest`, root user, secret-in-`ENV`, `COPY . .`, `chmod -R 777`, unpinned `pip install`) + 3 IAM (`Action:"*"`, `Resource:"*"`, no `Condition`), each found **and** fixed.

Single biggest-risk misconfig: the **hard-coded `AWS_SECRET_ACCESS_KEY` in `ENV`**, combined with the `Action:"*"/Resource:"*"` IAM policy. It needs **zero exploitation** — anyone who pulls the image reads a live AWS key straight from `docker inspect` — and because the matching policy is full-admin, that single leaked key is immediate, total account takeover. A hard-coded live credential is worse than a code flaw because there's no bug to trigger: possession of the artifact *is* the compromise.

Part 4 — Reflection

1. **Mapping.** For three findings, write the line of the artifact → CWE → OWASP 2025 A02, and the one-line fix.

   A:

Artifact line	CWE	OWASP 2025	One-line fix
`Dockerfile.insecure` — `FROM python:latest`	CWE-1104 / CWE-16	A02 Security Misconfiguration	Pin to `@sha256:` digest + slim/distroless base

Artifact line	CWE	OWASP 2025	One-line fix
`Dockerfile.insecure` — `ENV API_TOKEN=sk_live_...`	CWE-798 / CWE-200	A02 Security Misconfiguration	Remove from image; inject at runtime via secrets manager
`iam-policy-insecure.json` — `"Action":"*"` / `"Resource":"*"`	CWE-269 / CWE-732	A02 Security Misconfiguration	Scope to `s3:GetObject` on one bucket ARN + prefix, add `Condition`

2. **Real incident.** Briefly describe a real breach caused by a cloud/container misconfiguration. What single control from this lab would have prevented it?

   A: The **2019 Capital One breach** (~106 million customers) was a textbook A02 cloud misconfiguration: an SSRF against a misconfigured WAF on an EC2 instance let the attacker reach the instance metadata service and steal the attached **IAM role's credentials, which were over-permissive** — they could list and read ~100M records across many S3 buckets. It maps to **CWE-732 / CWE-269** (over-broad permissions and resources), exactly the `iam-policy-insecure.json` problem. The single control from this lab that would have prevented (or drastically limited) it is **least-privilege IAM**: had the role been scoped to only the specific bucket/prefix the app needed, with conditions, the stolen credentials would have read one prefix — not the entire customer data lake — turning a catastrophic breach into a contained one.

3. **Best mitigation.** Of everything you did, which control gives the most risk reduction per unit of effort, and why? Argue for least-privilege IAM vs. distroless+non-root.

   A: Least-privilege IAM gives the most risk reduction per unit of effort. It's a single policy edit (low effort) that **caps the blast radius of every other failure** — a stolen key, an SSRF, a compromised dependency, or an insider can only touch what the identity was scoped to, so one change bounds the damage across the whole system (the difference between Capital One's 100M records and a single prefix). **Distroless + non-root** is excellent and nearly free too, but it's narrower: it reduces the *likelihood and ease* of container compromise and local privilege escalation — it protects the runtime, not the cloud data behind it. For a cloud-backed app where the crown jewels are data reached through IAM, the IAM scoping is the control standing between a foothold and a full-account breach, so it edges out distroless on per-effort leverage. In practice you do both — they're complementary layers of defense in depth — but if I could apply only one, it's least-privilege IAM.

Grading rubric (100)

Component	Points	What earns full marks
Part 2 — Lecture questions	20	All 5 answered correctly with CWE/A02 reasoning
Part 3 — Tasks + evidence	40	Tasks 0–5 complete; commands, outputs, and screenshots present

Component	Points	What earns full marks
Defense (Task 4 hardening)	25	Hardened Dockerfile + least-priv IAM shown to reduce findings; per-fix justification
Part 4 — Reflection	15	Accurate mapping, relevant incident, well-argued best mitigation

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted. *The stamp runs in the host shell where the scan is issued, so `whoami` reads my account; Trivy runs in a throwaway `--rm` container and appears in the same transcript.*
- **Personalized flag (if this lab issues one):** This lab issues **no per-student flag** — the 9 "flags" are the 9 misconfigurations found and fixed (6 container + 3 IAM); the graded evidence is the before/after Trivy scan and the defect→fix mappings. *Flags are unique per student — submitting another student's flag is a violation. How to submit: **learn.zcr.ai/submit** (full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*

    1. What did you do, and **why did the vulnerability work**?

        A: I scanned the two Dockerfiles with Trivy, mapped each planted defect to its CWE, reviewed the full-admin IAM policy, and rebuilt both to least privilege / hardened. The misconfigurations "work" because a container image is just readable layers plus metadata, so a baked `ENV` secret leaks to anyone with the image, an unpinned `:latest` base drifts and can't be audited, `chmod 777` + root means any foothold owns the code, and an `Action:"*"/Resource:"*"` policy hands any leaked credential the whole account.

    2. **Why does your fix actually stop it** — and what could still break it?

        A: The hardened build pins the base by digest, keeps secrets out of the image (runtime injection), copies only `app.py` behind a `.dockerignore`, and runs non-root on distroless, while the IAM rewrite scopes actions/resources to one bucket prefix with a condition — so there's no secret to recover, no drift, no root foothold, and a leaked credential reaches almost nothing. What could still break it: a secret handed in at runtime can still leak if the orchestrator's secret store is misconfigured; distroless shrinks but doesn't eliminate CVEs, so the digest still needs periodic re-pinning and scanning; and IAM least privilege only holds if new features don't quietly re-broaden the policy — scope creep reopens CWE-269/732.

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

AI answer (fix for the baked-in secret):

```
FROM python:latest
ARG API_TOKEN
ENV API_TOKEN=$API_TOKEN
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
USER 1000
CMD ["python", "app.py"]
# build with: docker build --build-arg API_TOKEN=sk_live_... -t app .
```

> "Passing the token as a `--build-arg` instead of hard-coding it keeps the secret out of the image, and `USER 1000` drops root."

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

   A: The core claim is **false and dangerous**. `ARG API_TOKEN` + `ENV API_TOKEN=$API_TOKEN` does **not** keep the secret out of the image: the build arg is recorded and visible via `docker history --no-trunc`, and because it's assigned to an `ENV`, the value is baked into the image config and shows up in `docker inspect` (`Config.Env`) in clear text — exactly like the hard-coded version, so **CWE-798/200 are not fixed at all**. On top of that, `FROM python:latest` is still unpinned (CWE-1104/CWE-16), `COPY . .` still leaks the whole context (CWE-538), the `pip install` is still unpinned, and `USER 1000` is a guess — uid 1000 may not exist in the image, and the build still runs as root. It fixes essentially nothing while *sounding* like a secrets fix.

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

   Correct version (secret never in the image):

```
FROM python:3.11-slim@sha256:<pinned-digest> AS build
WORKDIR /app
COPY requirements.txt .
RUN python -m venv /venv && /venv/bin/pip install --no-cache-dir -r
requirements.txt
COPY app.py .

FROM gcr.io/distroless/python3-debian12@sha256:<pinned-digest>
WORKDIR /app
COPY --from=build /venv /venv
COPY --from=build /app/app.py /app/app.py
ENV PATH="/venv/bin:$PATH" PYTHONPATH="/venv/lib/python3.11/site-packages"
USER 65532:65532
CMD ["app.py"]
```

plus a `.dockerignore` (`.git`, `.env`, keys …) and the token supplied **only at runtime**: `docker run -e API_TOKEN="$API_TOKEN" week13-hardened:lab`. **Verified:** `docker history --no-trunc` and `docker inspect` on the built image show **no `API_TOKEN` anywhere** (no ENV, no ARG, no layer), and `trivy config` reports 0 findings. **Why the AI was insufficient:** it confused "not a string literal" with "not in the image" — build args and ENV both persist in the image, so the secret still leaks; and it left the base unpinned, the context fully copied, and root intact. A real fix keeps the secret out of *every* layer (runtime injection only) and hardens the base/user/copy at the same time.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

A: The insecure Dockerfile builds a shipping box for the app, but it prints a real password on the label (the `ENV` secret), grabs a base layer that can silently change under you (`latest`), tosses in everything in the folder including hidden secrets (`COPY . .`), makes every file editable by anyone (`chmod 777`), and runs the app as the all-powerful root user. It's exploitable because a Docker image is just a stack of readable layers plus a metadata file — anyone who gets the image can run `docker history` or `docker inspect` to read the baked-in secret **without even starting it**, and if the running app is ever hacked it's already root on world-writable code, so the attacker owns the container instantly. The cloud side is the same idea: the IAM policy hands the app a master key (`*` on `*`), so one stolen credential can touch the entire account instead of just the one bucket it needed.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

Final prompt: "Rewrite this Dockerfile so the secret is **not present anywhere in the built image** — not in `ENV`, not in a build `ARG`, not in any layer (it must not appear in `docker history --no-trunc` or `docker inspect`). The token must be provided **only at container runtime**. Also: pin the base image to an `@sha256:` digest (don't use `:latest`), copy only `app.py` (assume a `.dockerignore`), and run as a non-root user. Return the Dockerfile plus the `docker run` command that supplies the token at runtime." **Verified result:** the AI returned a digest-pinned, multi-stage, distroless Dockerfile with **no `ENV`/`ARG` for the token**, `USER 65532:65532`, and `docker run -e API_TOKEN="$API_TOKEN" …` for runtime injection. Re-checked the built image: `docker history --no-trunc` and `docker inspect` show the token nowhere, and `trivy config` is clean. Naming "not in a build ARG" and "must not appear in `docker history`/`inspect`" in the prompt is exactly what stopped the AI from regressing to the insecure `--build-arg` answer from the Audit section. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*